

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА 1

По дисциплине Промышленная маршрутизация и управление трафиком

Тема работы Развертывание тестовой сети связи с использованием ContainerLab

Обучающийся Рекун Ирина Данииловна

Факультет Факультет прикладной информатики

Группа К3220

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Рекун И.Д.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Аминов Н.С.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 ХОД РАБОТЫ.....	4
1.1 Подготовка среды выполнения лабораторной работы.....	4
1.2 Формирование сети связи	6
1.3 Конфигурация сетевых устройств.....	9
ЗАКЛЮЧЕНИЕ	14

ВВЕДЕНИЕ

В данной лабораторной работе выполняется подготовка среды для развертывания виртуальной сети связи. Для этого используется технология контейнеризации Docker и инструмент ContainerLab, предназначенный для моделирования сетевой инфраструктуры.

Цель - Ознакомиться с инструментом ContainerLab и методами работы с ним, изучить работу VLAN, IP адресации и т.д.

1 ХОД РАБОТЫ

1.1 Подготовка среды выполнения лабораторной работы

Для выполнения лабораторной работы была использована среда Ubuntu, установленная с помощью WSL (Windows Subsystem for Linux).

После установки Ubuntu была выполнена установка программного пакета Docker, который представляет собой платформу для контейнеризации приложений и позволяет запускать виртуальные сетевые устройства в изолированной среде. После установки Docker была выполнена проверка его версии с помощью команды терминала. (рисунок 1).

```
garriet@LAPTOP-4D6JQQMG:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~22.04.1
garriet@LAPTOP-4D6JQQMG:~$
```

Рисунок 1 — Проверка установленной версии Docker

Перед клонированием репозитория была выполнена проверка установленной утилиты make (рисунок 2).

```
garriet@LAPTOP-4D6JQQMG:~$ make --version
GNU Make 4.3
Эта программа собрана для x86_64-pc-linux-gnu
Copyright (C) 1988-2020 Free Software Foundation, Inc.
Лицензия GPLv3+: GNU GPL версии 3 или новее <http://gnu.org/licenses/gpl.html>
Это свободное программное обеспечение: вы можете свободно изменять его и
распространять. НЕТ НИКАКИХ ГАРАНТИЙ вне пределов, допустимых законом.
```

Рисунок 2 — Проверка установленной версии утилиты make

На следующем этапе был загружен репозиторий vrnetlab с платформы GitHub с помощью команды git clone, а также была создана директория vrnetlab, содержащая файлы проекта (рисунок 3). Далее был выполнен переход в данную директорию и просмотр её содержимого

```
garriet@LAPTOP-4D6JQQMG:~$ git clone https://github.com/hellt/vrnetlab.git
Клонирование в «vrnetlab»...
remote: Enumerating objects: 6272, done.
remote: Counting objects: 100% (1072/1072), done.
remote: Compressing objects: 100% (141/141), done.
remote: Total 6272 (delta 977), reused 937 (delta 929), pack-reused 5200 (from 2)
Получение объектов: 100% (6272/6272), 4.03 МиБ | 10.78 МиБ/с, готово.
Определение изменений: 100% (3747/3747), готово.
```

Рисунок 3 — Клонирование репозитория vrnetlab

После перехода в директорию проекта был выполнен просмотр её содержимого (рисунок 4). В данной директории находятся различные каталоги, предназначенные для запуска сетевых операционных систем. Среди них присутствует каталог mikrotik, содержащий файлы для запуска операционной системы RouterOS.

```
garriet@LAPTOP-4D6JQQMG:~$ cd vrnetlab
garriet@LAPTOP-4D6JQQMG:~/vrnetlab$ ls
arista      CONTRIBUTING.md  freebsd        makefile.include  openwrt         ubuntu
aruba       dell             hp             makefile-install.include  paloalto        uv.lock
build-base-image.sh  dev-notes.md    huawei          makefile-sanity.include  pyproject.toml  vrnetlab-base.dockerfile
cisco       extreme          ipinfusion     mikrotik           README.md
CODE_OF_CONDUCT.md  f5_bigip        juniper        nokia              sonic
common       fortinet         LICENSE        openbsd            spirent
```

Рисунок 4 — Просмотр содержимого директории vrnetlab

Была выполнена подготовка образа операционной системы RouterOS. Для этого был выполнен переход в каталог mikrotik/routeros, содержащий файлы для сборки контейнера с виртуальным маршрутизатором MikroTik. В данный каталог был добавлен файл виртуального диска chr-6.47.9.vmdk, содержащий образ операционной системы RouterOS (рисунок 5).

```
garriet@LAPTOP-4D6JQQMG:~/vrnetlab/mikrotik$ cd routeros
garriet@LAPTOP-4D6JQQMG:~/vrnetlab/mikrotik/routeros$ cp /mnt/c/Users/denik/Downloads/chr-6.47.9.vmdk .
```

Рисунок 5 — Добавление файла образа виртуального диска в каталог routeros

После добавления файла виртуального диска chr-6.47.9.vmdk, была выполнена сборка контейнера с использованием команды make docker-image. После успешной сборки контейнера была выполнена проверка созданного образа с помощью команды docker images | grep mikrotik. Так, в списке docker-образов есть образ vrnetlab/mikrotik-routeros версии 6.47.9, что подтверждает успешную сборку контейнера (рисунок 6).

```
=> => naming to docker.io/vrnetlab/mikrotik-routeros:6.47.9-amd64
Adding default tag 6.47.9 pointing to 6.47.9-amd64 (matches host platform: amd64)
make[1]: выход из каталога «/home/garriet/vrnetlab/mikrotik/routeros»
make[1]: вход в каталог «/home/garriet/vrnetlab/mikrotik/routeros»
Makefile:28: предупреждение: переопределение способа для цели «docker-build-extra»
../makefile.include:56: предупреждение: старый способ для цели «docker-build-extra» игнорируются
Makefile:36: предупреждение: переопределение способа для цели «docker-push-image-extra»
../makefile.include:66: предупреждение: старый способ для цели «docker-push-image-extra» игнорируются
--> Cleaning docker build context
rm -f docker/*.qcow2* docker/*.tgz* docker/*.vmdk* docker/*.iso docker/*.xml docker/*.bin
rm -f docker/healthcheck.py docker/vrnetlab.py
make[1]: выход из каталога «/home/garriet/vrnetlab/mikrotik/routeros»
garriet@LAPTOP-4D6JQQMG:~/vrnetlab/mikrotik/routeros$ docker images | grep mikrotik
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
vrnetlab/mikrotik-routeros:6.47.9      78df34fd5bd3      1.62GB      0B
vrnetlab/mikrotik-routeros:6.47.9-amd64 78df34fd5bd3      1.62GB      0B
```

Рисунок 6 — Сборка и проверка контейнера

После сборки контейнера с помощью специального скрипта из официального репозитория проекта был установлен ContainerLab (рисунок 7).

[illegible]

Рисунок 7 — Установка ContainerLab

1.2 Формирование сети связи

Необходимо было создать трёхуровневую сеть связи классического предприятия. Для этого требовалось нарисовать схему связи, создать все устройства, указанные на схеме, и настроить соединения между ними.

Так, сначала была составлена схема связи. На схеме показаны маршрутизатор, центральный коммутатор, два коммутатора доступа и два пользовательских компьютера. Также на схеме указаны соединения между устройствами и используемые VLAN. Схема связи представлена на рисунке 8

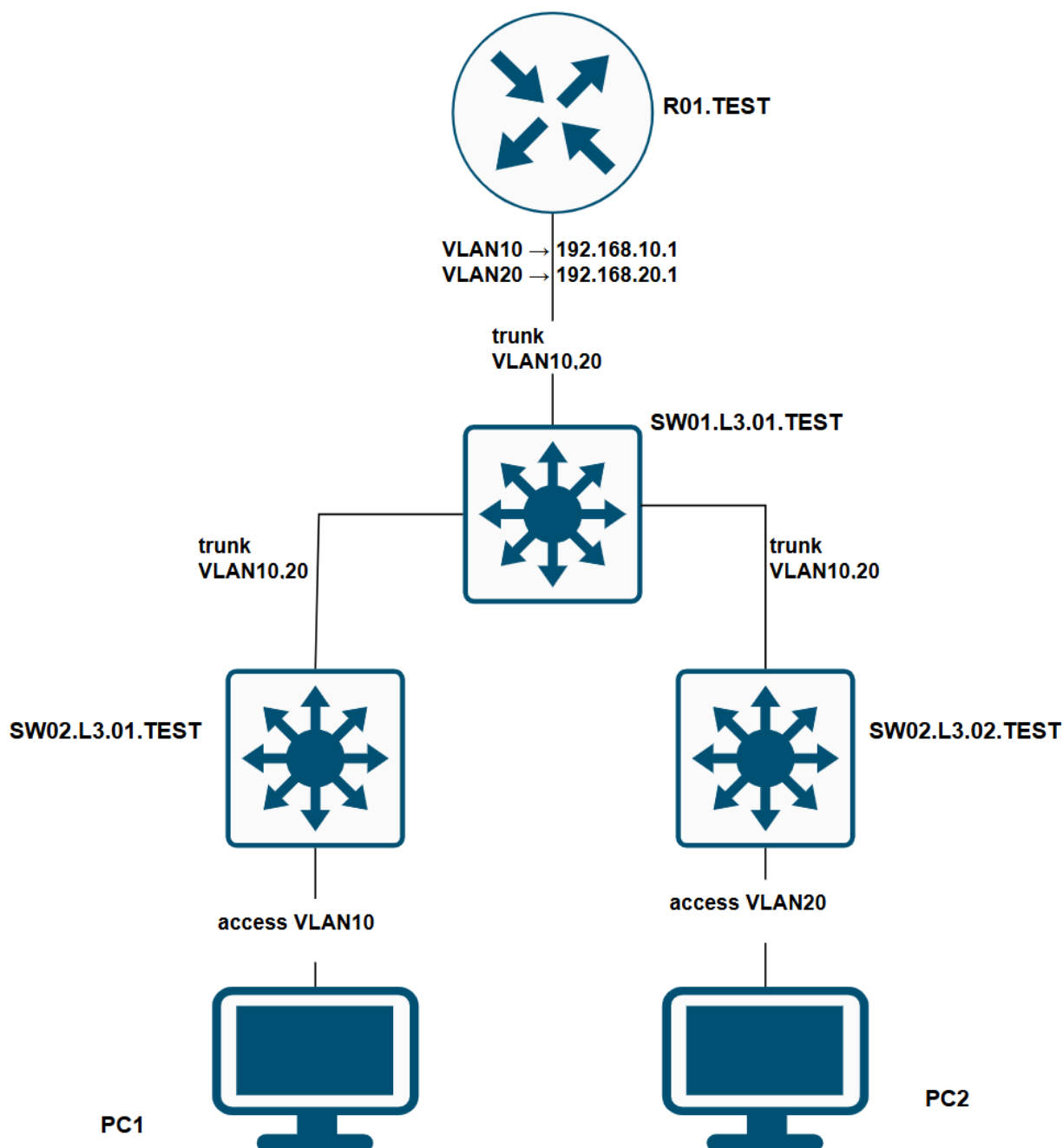
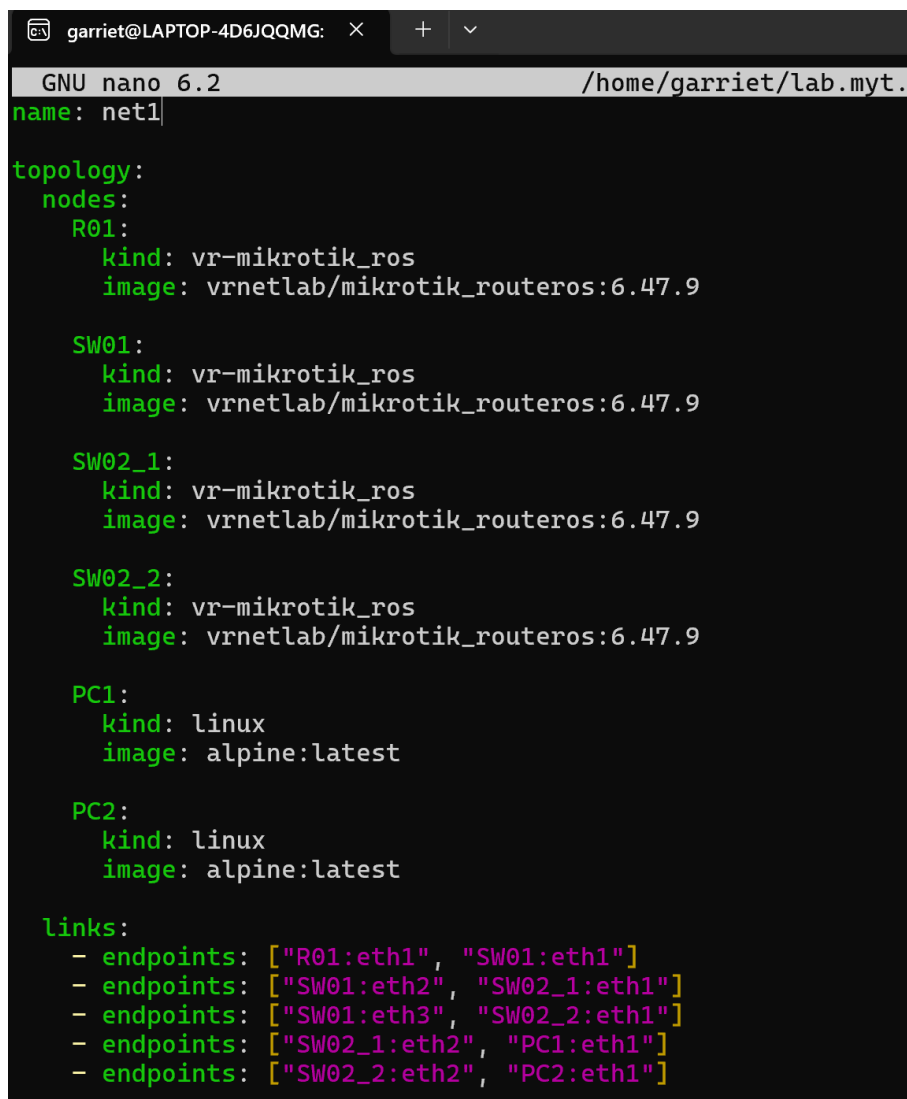


Рисунок 8 — Установка ContainerLab

Сначала был подготовлен файл конфигурации топологии `topology.clab.yml`. В этом файле описывается структура сети: какие устройства используются и как они соединены между собой. В топологии были указаны следующие устройства: маршрутизатор R01, коммутатор распределительного уровня SW01, два коммутатора доступа, а также два конечных устройства PC1 и PC2. Также в конфигурации были заданы соединения

между устройствами в соответствии со схемой сети. Конфигурация файла топологии представлена на рисунке 9.



```
garriet@LAPTOP-4D6JQQMG: × + ▾
GNU nano 6.2 /home/garriet/lab.myt.
name: net1

topology:
  nodes:
    R01:
      kind: vr-mikrotik_ros
      image: vrnetlab/mikrotik_routeros:6.47.9

    SW01:
      kind: vr-mikrotik_ros
      image: vrnetlab/mikrotik_routeros:6.47.9

    SW02_1:
      kind: vr-mikrotik_ros
      image: vrnetlab/mikrotik_routeros:6.47.9

    SW02_2:
      kind: vr-mikrotik_ros
      image: vrnetlab/mikrotik_routeros:6.47.9

    PC1:
      kind: linux
      image: alpine:latest

    PC2:
      kind: linux
      image: alpine:latest

  links:
    - endpoints: ["R01:eth1", "SW01:eth1"]
    - endpoints: ["SW01:eth2", "SW02_1:eth1"]
    - endpoints: ["SW01:eth3", "SW02_2:eth1"]
    - endpoints: ["SW02_1:eth2", "PC1:eth1"]
    - endpoints: ["SW02_2:eth2", "PC2:eth1"]
```

Рисунок 9 — Файл конфигурации

После создания файла топологии была выполнена команда запуска лабораторной сети: `containerlab deploy -t topology.clab.yml`. Данная команда запускает процесс развёртывания сети в ContainerLab. В ходе выполнения создаются Docker-контейнеры для всех устройств, указанных в файле топологии, и автоматически настраиваются соединения между ними.

В результате были созданы контейнеры для маршрутизатора R01, трех коммутаторов, а также для конечных устройств PC1 и PC2. После запуска контейнеры перешли в состояние `running`, что подтверждает успешное развёртывание сети. Результат выполнения представлен на рисунке 10.


```

garriet@LAPTOP-4D6JQQMG:~/vrnetlab/mikrotik/routeros$ nano ~/lab.myt.1/topology.clab.yml
garriet@LAPTOP-4D6JQQMG:~/vrnetlab/mikrotik/routeros$ cd ~/lab.myt.1
containerlab deploy -t topology.clab.yml
01:59:45 INFO Containerlab started version=0.74.0
01:59:45 INFO Parsing & checking topology file=topology.clab.yml
01:59:45 WARN Node name will not resolve via DNS name=clab-net1-SW02_1 reason="name contains invalid characters
such as '.' and/or '_' "
01:59:45 WARN Node name will not resolve via DNS name=clab-net1-SW02_2 reason="name contains invalid characters
such as '.' and/or '_' "
01:59:45 INFO Creating lab directory path=/home/garriet/lab.myt.1/clab-net1
01:59:45 INFO Creating container name=PC1
01:59:45 INFO Creating container name=SW01
01:59:45 INFO Creating container name=SW02_1
01:59:45 INFO Creating container name=SW02_2
01:59:45 INFO Creating container name=PC2
01:59:45 INFO Creating container name=R01
01:59:46 INFO Created link: SW02_1:eth2 - - - - - PC1:eth1
01:59:46 INFO Created link: SW02_2:eth2 - - - - - PC2:eth1
01:59:46 INFO Created link: R01:eth1 - - - - - SW01:eth1
01:59:47 INFO Created link: SW01:eth2 - - - - - SW02_1:eth1
01:59:47 INFO Created link: SW01:eth3 - - - - - SW02_2:eth1
01:59:47 INFO Adding host entries path=/etc/hosts
01:59:47 INFO Adding SSH config for nodes path=/etc/ssh/ssh_config.d/clab-net1.conf

```

Name	Kind/Image	State	IPv4/6 Address
clab-net1-PC1	linux alpine:latest	running	172.20.20.4 3fff:172:20:20::4
clab-net1-PC2	linux alpine:latest	running	172.20.20.6 3fff:172:20:20::6
clab-net1-R01	vr-mikrotik_ros vrnetlab/mikrotik_routeros:6.47.9	running (health: starting)	172.20.20.5 3fff:172:20:20::5
clab-net1-SW01	vr-mikrotik_ros vrnetlab/mikrotik_routeros:6.47.9	running (health: starting)	172.20.20.7 3fff:172:20:20::7
clab-net1-SW02_1	vr-mikrotik_ros vrnetlab/mikrotik_routeros:6.47.9	running (health: starting)	172.20.20.2 3fff:172:20:20::2
clab-net1-SW02_2	vr-mikrotik_ros vrnetlab/mikrotik_routeros:6.47.9	running (health: starting)	172.20.20.3 3fff:172:20:20::3

Рисунок 10 — Развёртывание сети с помощью ContainerLab

1.3 Конфигурация сетевых устройств

После успешного запуска лабораторной сети было выполнено подключение к маршрутизатору R1 по протоколу SSH для дальнейшей настройки сети. В интерфейсе ether1 были созданы VLAN1 и VLAN2 для разделения сети на два сегмента. VLAN1 предназначен для устройства PC1, VLAN2 — для PC2. После каждому из них были назначены IP-адреса, которые используются как шлюзы для соответствующих подсетей. Затем были созданы пулы адресов для автоматической выдачи IP-адресов устройствам в этих подсетях. На основе созданных пулов были настроены два DHCP-сервера, которые будут автоматически выдавать IP-адреса устройствам в соответствующих VLAN. После этого были добавлены сети DHCP, в которых указываются подсети и шлюзы для клиентов. Результат изображен на рисунке 11.

```
[admin@R01.TEST] > /interface vlan print
Flags: X - disabled, R - running
#   NAME                               MTU ARP           VLAN-ID INTERFACE
0 R vlan10                             1500 enabled        10 ether2
1 R vlan20                             1500 enabled        20 ether2
```

Рисунок 11 — Созданные VLAN

На рисунке 12 представлена конфигурация маршрутизатора R01.TEST. В конфигурации представлено создание VLAN-интерфейсов VLAN10 и VLAN20 на физическом интерфейсе, назначение IP-адресов для каждой подсети, а также настройка DHCP-серверов для автоматической выдачи IP-адресов клиентским устройствам.

```
[admin@R01] > /system identity set name=R01.TEST
[admin@R01.TEST] > /user set [find name=admin] password=hjenth
[admin@R01.TEST] >
[admin@R01.TEST] > /interface vlan
[admin@R01.TEST] /interface vlan> add name=vlan10 vlan-id=10 interface=ether2
[admin@R01.TEST] /interface vlan> add name=vlan20 vlan-id=20 interface=ether2
[admin@R01.TEST] /interface vlan>
[admin@R01.TEST] /interface vlan> /ip address
[admin@R01.TEST] /ip address> add address=192.168.10.1/24 interface=vlan10
[admin@R01.TEST] /ip address> add address=192.168.20.1/24 interface=vlan20
[admin@R01.TEST] /ip address>
[admin@R01.TEST] /ip address> /ip pool
[admin@R01.TEST] /ip pool> add name=pool10 ranges=192.168.10.100-192.168.10.253
[admin@R01.TEST] /ip pool>
[admin@R01.TEST] /ip pool> /ip dhcp-server
[admin@R01.TEST] /ip dhcp-server> add name=dhcp10 interface=vlan10 address-pool=pool10
[admin@R01.TEST] /ip dhcp-server>
[admin@R01.TEST] /ip dhcp-server> /ip dhcp-server network
[admin@R01.TEST] /ip dhcp-server network> add address=192.168.10.0/24 gateway=192.168.10.1
[admin@R01.TEST] /ip dhcp-server network>
[admin@R01.TEST] /ip dhcp-server network> /ip dhcp-server enable dhcp10
```

Рисунок 12 — Конфигурация маршрутизатора R01.TEST

После чего написана конфигурация центрального коммутатора SW01.L3.01.TEST. Был создан bridge-интерфейс с включенной фильтрацией VLAN, добавление портов в bridge и выполнена настройка таблицы VLAN, обеспечивающей передачу тегированного трафика VLAN10 и VLAN20 между маршрутизатором и коммутаторами доступа.

```

[admin@SW01.L3.01.TEST] >
[admin@SW01.L3.01.TEST] > /interface bridge
[admin@SW01.L3.01.TEST] /interface bridge> add name=bridge1 vlan-filtering=yes
[admin@SW01.L3.01.TEST] /interface bridge>
[admin@SW01.L3.01.TEST] /interface bridge> /interface bridge port
[admin@SW01.L3.01.TEST] /interface bridge port> add bridge=bridge1 interface=ether2
[admin@SW01.L3.01.TEST] /interface bridge port> add bridge=bridge1 interface=ether4
[admin@SW01.L3.01.TEST] /interface bridge port>
[admin@SW01.L3.01.TEST] /interface bridge port> /interface bridge vlan
[admin@SW01.L3.01.TEST] /interface bridge vlan> add bridge=bridge1 vlan-ids=10 tagged=ether2,ether3
[admin@SW01.L3.01.TEST] /interface bridge vlan> add bridge=bridge1 vlan-ids=20 tagged=ether2,ether4
[admin@SW01.L3.01.TEST] /interface bridge vlan>

```

Рисунок 13 — Конфигурация центрального коммутатора

На рисунках 14 и 15 отображена конфигурация коммутаторов доступа SW02.L3.01.TEST и SW02.L3.02.TEST. Необходимо было выполнить: настройку bridge-интерфейса, добавить порты и конфигурацию VLAN, при которой порт подключения пользовательского устройства PC1 работает как access-порт VLAN10, а порт соединения с центральным коммутатором работает в режиме trunk. На коммутаторе SW02.L3.01.TEST порт подключения PC1 был назначен в VLAN10, а на коммутаторе SW02.L3.02.TEST порт подключения PC2 — в VLAN20.

```

[admin@SW02.L3.01.TEST] > /interface bridge
[admin@SW02.L3.01.TEST] /interface bridge> add name=bridge1 vlan-filtering=yes
[admin@SW02.L3.01.TEST] /interface bridge>
[admin@SW02.L3.01.TEST] /interface bridge> /interface bridge port
[admin@SW02.L3.01.TEST] /interface bridge port> add bridge=bridge1 interface=ether2
[admin@SW02.L3.01.TEST] /interface bridge port> add bridge=bridge1 interface=ether3 pvid=10
[admin@SW02.L3.01.TEST] /interface bridge port>
[admin@SW02.L3.01.TEST] /interface bridge port> /interface bridge vlan
[admin@SW02.L3.01.TEST] /interface bridge vlan> add bridge=bridge1 vlan-ids=10 tagged=ether2 untagged=ether3
[admin@SW02.L3.01.TEST] /interface bridge vlan> add bridge=bridge1 vlan-ids=20 tagged=ether2
[admin@SW02.L3.01.TEST] /interface bridge vlan>
Connection to 172.20.20.6 closed.

```

Рисунок 14 — Настройка коммутатора доступа SW02.L3.01.TEST

```

[admin@SW02.L3.02.TEST] > /interface bridge
[admin@SW02.L3.02.TEST] /interface bridge> add name=bridge1 vlan-filtering=yes
[admin@SW02.L3.02.TEST] /interface bridge>
[admin@SW02.L3.02.TEST] /interface bridge> /interface bridge port
[admin@SW02.L3.02.TEST] /interface bridge port> add bridge=bridge1 interface=ether2
[admin@SW02.L3.02.TEST] /interface bridge port> add bridge=bridge1 interface=ether3 pvid=20
[admin@SW02.L3.02.TEST] /interface bridge port>
[admin@SW02.L3.02.TEST] /interface bridge port> /interface bridge vlan
[admin@SW02.L3.02.TEST] /interface bridge vlan> add bridge=bridge1 vlan-ids=10 tagged=ether2
[admin@SW02.L3.02.TEST] /interface bridge vlan> add bridge=bridge1 vlan-ids=20 tagged=ether2 untagged=ether3

```

Рисунок 15 — Настройка коммутатора доступа SW02.L3.02.TEST

Для проверки локальной связности были выполнены команды проверки сетевых параметров и связности на пользовательских узлах PC1 и PC2.

На узле PC1 была выполнена команда `ip addr show eth1`, в результате чего было установлено, что устройство получило IP-адрес 192.168.10.253/24

из сети VLAN10. Команда `ip route show` показала наличие маршрута по умолчанию через шлюз 192.168.10.1, который соответствует интерфейсу маршрутизатора R01. Проверка связности с маршрутизатором с помощью команды `ping 192.168.10.1` показала успешную передачу пакетов без потерь. Также была выполнена проверка связи с узлом PC2, расположенным в другой VLAN. Команда `ping 192.168.20.253` показала успешную передачу пакетов, что подтверждает корректную работу межвлановой маршрутизации. Процесс и результаты проверки представлены на рисунке 16.

```
garriet@LAPTOP-4D6JQQMG:~/lab.myt.1$ docker exec -it clab-net1-PC1 sh
/ # ip addr show eth1
195: eth1@if194: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 9500 qdisc noqueue state UP
    link/ether aa:c1:ab:a2:57:e4 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.253/24 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::a8c1:abff:fea2:57e4/64 scope link
        valid_lft forever preferred_lft forever
/ # ip route show
default via 192.168.10.1 dev eth1 metric 395
172.20.20.0/24 dev eth0 scope link src 172.20.20.2
192.168.10.0/24 dev eth1 scope link src 192.168.10.253
/ # ping -c 4 192.168.10.1
PING 192.168.10.1 (192.168.10.1): 56 data bytes
64 bytes from 192.168.10.1: seq=0 ttl=64 time=4.097 ms
64 bytes from 192.168.10.1: seq=1 ttl=64 time=2.185 ms
64 bytes from 192.168.10.1: seq=2 ttl=64 time=4.457 ms
^C
--- 192.168.10.1 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 2.185/3.579/4.457 ms
/ # ping -c 4 192.168.20.253
PING 192.168.20.253 (192.168.20.253): 56 data bytes
64 bytes from 192.168.20.253: seq=0 ttl=63 time=12.607 ms
64 bytes from 192.168.20.253: seq=1 ttl=63 time=4.072 ms
64 bytes from 192.168.20.253: seq=2 ttl=63 time=4.940 ms
64 bytes from 192.168.20.253: seq=3 ttl=63 time=4.237 ms

--- 192.168.20.253 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
```

Рисунок 16 — Настройка коммутатора доступа SW02.L3.01.TEST

На узле PC2 была выполнена аналогичная проверка. Устройство получило IP-адрес 192.168.20.253/24 из сети VLAN20. В таблице маршрутизации был обнаружен маршрут по умолчанию через шлюз 192.168.20.1. Проверка связи с маршрутизатором командой `ping 192.168.20.1` показала отсутствие потерь пакетов, что подтверждает корректную работу сети. Процесс и результаты проверки представлены на рисунке 17

```

garriet@LAPTOP-4D6JQQMG:~/lab.myt.1$ docker exec -it clab-net1-PC2 sh
/ # ip addr show eth1
201: eth1@if200: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 9500 qdisc noqueue state UP
    link/ether aa:c1:ab:92:e1:ca brd ff:ff:ff:ff:ff:ff
    inet 192.168.20.253/24 scope global eth1
        valid_lft forever preferred_lft forever
    inet6 fe80::a8c1:abff:fe92:e1ca/64 scope link
        valid_lft forever preferred_lft forever
/ # ip route show
ping -c 4 192.168.20.1 default via 192.168.20.1 dev eth1 metric 401
172.20.20.0/24 dev eth0 scope link src 172.20.20.5
192.168.20.0/24 dev eth1 scope link src 192.168.20.253
/ # ping -c 4 192.168.20.1
PING 192.168.20.1 (192.168.20.1): 56 data bytes
64 bytes from 192.168.20.1: seq=0 ttl=64 time=4.236 ms
64 bytes from 192.168.20.1: seq=1 ttl=64 time=2.214 ms
64 bytes from 192.168.20.1: seq=2 ttl=64 time=1.970 ms
64 bytes from 192.168.20.1: seq=3 ttl=64 time=3.267 ms

--- 192.168.20.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 1.970/2.921/4.236 ms

```

Рисунок 17 — Настройка коммутатора доступа SW02.L3.01.TEST

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы был изучен инструмент ContainerLab и принципы построения тестовой сети связи. Была создана схема сети, настроены маршрутизатор и коммутаторы, а также реализована работа VLAN и DHCP. После выполнения настроек была проведена проверка сетевой связности между узлами сети. Результаты проверки показали корректную работу сети, межвлановой маршрутизации и получение IP-адресов по DHCP.